AUTOMATED THERMAL SCREENING AND ACCESS CONTROL SYSTEM

Smart Entry Control with IoT Monitoring

Idea and Motivation

The Idea

To Build a smart entry system that automatically checks both identity and body temperature before allowing access.

Scan RFID card > Check ID > Measure temperature > Allow or deny entry automatically.

All records are logged and viewable on a phone via WiFi in real time.

Another important advantage is efficiency. Manual attendance takes time and can lead to errors. Similarly, manual temperature checks require extra manpower. This system reduces human effort, minimizes contact, and speeds up the entire process.

Why We Built This

During COVID-19, every office, hospital and school needed manual thermal screening at entry — slow, error-prone and needed extra staff.

Our system replaces manual screening with an automated, contactless solution using low-cost electronics.

The main motivation behind this project is to create a low-cost and easily deployable solution. It can be installed at multiple entry points, making it highly useful for schools, colleges, offices, and even public institutions. Instead of having separate systems for attendance and health checks, this integrated approach simplifies the process

System Block Diagram



Pin Connection Table

Component	Pin / Wire	Arduino Pin	Note
MFRC522 RFID	SDA	Pin 10	SPI SS
MFRC522 RFID	SCK	Pin 13	SPI Clock
MFRC522 RFID	MOSI	Pin 11	SPI Data Out
MFRC522 RFID	MISO	Pin 12	SPI Data In
MFRC522 RFID	RST	Pin 9	Reset
MFRC522 RFID	3.3V / GND	3.3V / GND	Use 3.3V only!
LM35	VCC / GND	5V / GND	-
LM35	OUT	A0	Analog temp
LCD 16x2	RS / EN	Pin 2 / Pin 3	4-bit mode
LCD 16x2	D4-D7	Pin 4, 5, 6, 7	Data pins
LCD 16x2	V0 (Pin 3)	GND	Max contrast
LCD 16x2	A (Pin 15)	5V via 220Ω	Backlight
Green LED	Anode	A1 via 220Ω	Allow
Red LED	Anode	A2 via 220Ω	Deny
ESP8266	TX	Pin 8 (RX)	SoftSerial
ESP8266	RX	A3 via divider	1kΩ + 2kΩ
ESP8266	VCC	3.3V	NOT 5V!

CODE AND EXPLANATION:



```
#include <SPI.h>              // SPI communication library (used by RFID)
#include <MFRC522.h>          // RFID reader library
#include <LiquidCrystal.h>    // LCD display library

// Pin definitions (makes code readable)
#define RST_PIN   9           // RFID Reset pin
#define SS_PIN    10          // RFID SDA (Slave Select)
#define GREEN_LED A1          // Green LED pin
#define RED_LED   A2          // Red LED pin
#define TEMP_PIN  A0          // LM35 output pin

// Create RFID object with SS and RST pins
MFRC522 rfid(SS_PIN, RST_PIN);

// Create LCD object with pins: RS, EN, D4, D5, D6, D7
LiquidCrystal lcd(2, 3, 4, 5, 6, 7);

// Valid RFID card UID (only this card is allowed)
const char UID1[] = "21077980";

// Temperature limit (above this → access denied)
float tempLimit = 37.5;

void setup() {
  Serial.begin(9600);     // Start serial communication
  SPI.begin();            // Start SPI bus for RFID
  rfid.PCD_Init();        // Initialize RFID reader

  // Set LED pins as output
  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);

  // Turn OFF LEDs initially
  digitalWrite(GREEN_LED, LOW);
  digitalWrite(RED_LED, LOW);

  // Initialize LCD (16 columns, 2 rows)
  lcd.begin(16, 2);
  lcd.clear();

  // Display startup message
  lcd.print("  Scan Your ID  ");
  lcd.setCursor(0, 1);
  lcd.print(" System Ready.. ");
  delay(2000);            // Wait 2 seconds

  lcd.clear();
  lcd.print("  Scan Your ID  ");

  Serial.println("System Ready!");
}

void loop() {

  // Check if a new RFID card is present
  // If no card → exit loop and keep checking
  if (!rfid.PICC_IsNewCardPresent() || !rfid.PICC_ReadCardSerial()) return;

  // Convert UID bytes into a readable string
  String uid = "";
  for (byte i = 0; i < rfid.uid.size; i++) {
    // Add leading zero if needed (for proper format)
    if (rfid.uid.uidByte[i] < 0x10) uid += "0";

    // Convert byte to HEX and append
    uid += String(rfid.uid.uidByte[i], HEX);
  }

  uid.toUpperCase();      // Convert to uppercase for comparison
  Serial.println(uid);    // Print UID in serial monitor

  // Check if scanned UID matches stored UID
  if (uid != UID1) {

    //  WRONG CARD → deny access immediately (no temp check)
    lcd.clear();
    lcd.print("!! Wrong ID !!");
    lcd.setCursor(0, 1);
    lcd.print(" Access Denied ");

    Serial.println("Wrong ID");

    redLight();           // Turn ON red LED

    delay(3000);          // Show message for 3 seconds
```



```
    // Reset LCD
    lcd.clear();
    lcd.print("  Scan Your ID  ");

    // Stop communication with current card
    rfid.PICC_HaltA();
    rfid.PCD_StopCrypto1();

    delay(500);

    return;   //  stop further execution
  }

  //  VALID CARD
  lcd.clear();
  lcd.print("  ID Verified!  ");
  lcd.setCursor(0, 1);
  lcd.print(" Hello Person1! ");
  delay(2000);

  // Ask user to wait for temperature check
  lcd.clear();
  lcd.print(" Check Your Temp");
  lcd.setCursor(0, 1);
  lcd.print("  Please Wait.. ");
  delay(2000);

  // Read LM35 value and convert to temperature
  // Formula:
  // analogRead → 0-1023
  // Convert to voltage → (5.0 / 1023.0)
  // LM35 gives 10mV per °C → multiply by 100
  float temp = analogRead(TEMP_PIN) * (5.0 / 1023.0) * 100.0;

  // Convert temperature to string with 1 decimal
  String tempStr = String(temp, 1) + "C";

  lcd.clear();

  // Check temperature condition
  if (temp > tempLimit) {

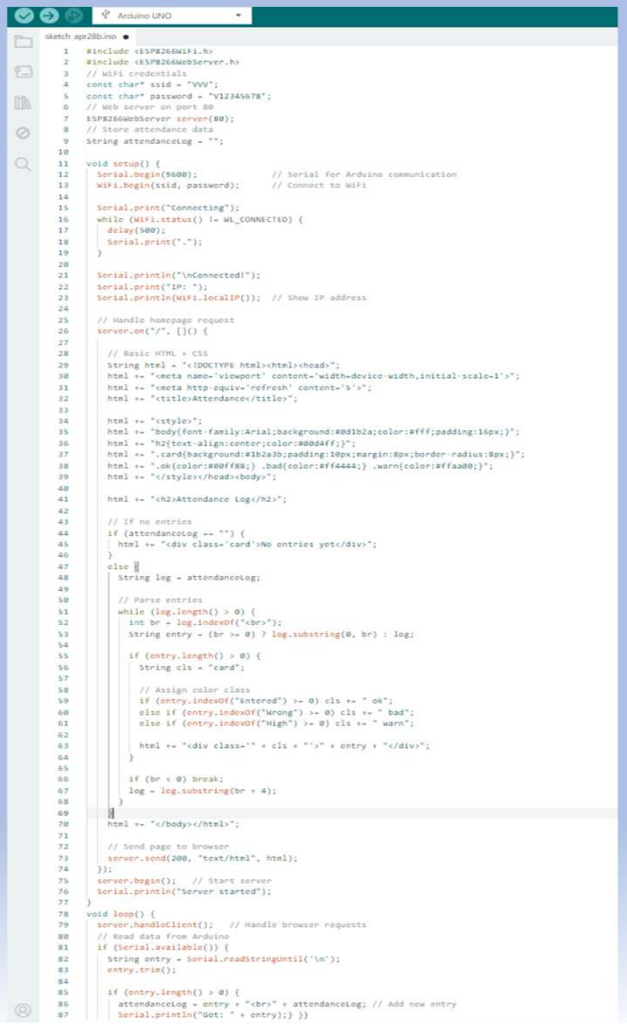
    // HIGH TEMPERATURE → deny access
    lcd.print("Person1         ");
    lcd.setCursor(0, 1);
    lcd.print(tempStr + " High! Deny ");

    Serial.println("High Temp");

    redLight();           // Turn ON red LED

  } else {

    // NORMAL TEMPERATURE → allow access
    lcd.print("Person1 OK!     ");
    lcd.setCursor(0, 1);
    lcd.print(tempStr + " Welcome!  ");
    Serial.println("Access Granted");
    greenLight();         // Turn ON green LED
  }
  delay(3000);
  // Reset LCD for next user
  lcd.clear();
  lcd.print("  Scan Your ID  ");
  // Stop communication with card
  rfid.PICC_HaltA();
  rfid.PCD_StopCrypto1();
  delay(500);}
// GREEN LED FUNCTION → Access Granted
void greenLight() {
  digitalWrite(GREEN_LED, HIGH);   // Turn ON green LED
  digitalWrite(RED_LED, LOW);      // Ensure red LED OFF
  delay(3000);                     // Wait 3 seconds
  digitalWrite(GREEN_LED, LOW);    // Turn OFF green LED
}
//  RED LED FUNCTION → Access Denied
void redLight() {
  digitalWrite(RED_LED, HIGH);     // Turn ON red LED
  digitalWrite(GREEN_LED, LOW);    // Ensure green LED OFF
  delay(3000);                     // Wait 3 seconds
  digitalWrite(RED_LED, LOW);      // Turn OFF red LED
}
```



```
#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>
// WiFi credentials
const char* ssid = "VVVV";
const char* password = "V12345678";
// Web server on port 80
ESP8266WebServer server(80);
// Store attendance data
String attendanceLog = "";

void setup() {
  Serial.begin(9600);               // Serial for Arduino communication
  WiFi.begin(ssid, password);       // Connect to WiFi

  Serial.print("Connecting");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("\nConnected!");
  Serial.print("IP: ");
  Serial.println(WiFi.localIP());  // Show IP address

  // Handle homepage request
  server.on("/", []() {

    // Basic HTML + CSS
    String html = "<!DOCTYPE html><html><head>";
    html += "<meta name='viewport' content='width=device-width,initial-scale=1'>";
    html += "<meta http-equiv='refresh' content='5'>";
    html += "<title>Attendance</title>";

    html += "<style>";
    html += "body{font-family:Arial;background:#0d1b2a;color:#fff;padding:16px;}";
    html += "h2{text-align:center;color:#00d4ff;}";
    html += ".card{background:#1b2a3b;padding:10px;margin:8px;border-radius:8px;}";
    html += ".ok{color:#00ff88;} .bad{color:#ff4444;} .warn{color:#ffaa00;}";
    html += "</style></head><body>";

    html += "<h2>Attendance Log</h2>";

    // If no entries
    if (attendanceLog == "") {
      html += "<div class='card'>No entries yet</div>";
    }
    else {
      String log = attendanceLog;

      // Parse entries
      while (log.length() > 0) {
        int br = log.indexOf("<br>");
        String entry = (br >= 0) ? log.substring(0, br) : log;

        if (entry.length() > 0) {
          String cls = "card";

          // Assign color class
          if (entry.indexOf("Entered") >= 0) cls += " ok";
          else if (entry.indexOf("Wrong") >= 0) cls += " bad";
          else if (entry.indexOf("High") >= 0) cls += " warn";

          html += "<div class='" + cls + "'>" + entry + "</div>";
        }

        if (br < 0) break;
        log = log.substring(br + 4);
      }
    }
    html += "</body></html>";

    // Send page to browser
    server.send(200, "text/html", html);
  });
  server.begin();   // Start server
  Serial.println("Server started");
}
void loop() {
  server.handleClient();   // Handle browser requests
  // Read data from Arduino
  if (Serial.available()) {
    String entry = Serial.readStringUntil('\n');
    entry.trim();

    if (entry.length() > 0) {
      attendanceLog = entry + "<br>" + attendanceLog; // Add new entry
      Serial.println("Got: " + entry);} }
```

Working Procedure of the Code:

The Arduino code follows a step-by-step logic flow when a card is scanned:

Step 1 — System Initialization

During setup(), the Arduino initializes the SPI bus, RFID module, LCD display, and LED pins. The LCD displays 'Scan Your ID' to indicate the system is ready. A 500ms delay ensures the LCD has sufficient time to power up before any commands are sent, preventing display artifacts.

Step 2 — RFID Card Scanning

In loop(), the code continuously polls the RFID reader using **PICC_IsNewCardPresent() and** PICC_ReadCardSerial(). When a card is detected, the UID bytes are read and converted to a hexadecimal string. The UID is compared against the registered UID stored in the program (UID1 = "21077900").

Step 3 — Identity Verification

If the scanned UID does not match the registered UID, the system immediately shows '!! Wrong ID !!' on the LCD, activates the Red LED for 3 seconds, and returns without proceeding to temperature measurement. If the UID matches, the LCD shows 'ID Verified! Hello Person1!' confirming valid identity.

Step 4 — Temperature Measurement

After identity verification, the LCD displays 'Check Your Temp – Please Wait' for 2 seconds, allowing the LM35 sensor to stabilize. The analog value from pin A0 is read using analogRead() and converted to Celsius using the formula: Temp = ADC_Value × (5.0 / 1023.0) × 100.0, since LM35 produces 10mV per degree Celsius.

Step 5 — Access Decision

The measured temperature is compared against the threshold of 37.5°C. If temperature exceeds the limit, the Red LED glows and LCD shows 'High Temp – Deny'. If temperature is within the safe range, the Green LED glows and LCD shows 'Person1 OK! – Welcome!'. After 3 seconds, the display resets to 'Scan Your ID'.

Working Procedure of the ESP8266 Code:

The NodeMCU code manages **WiFi communication and displays attendance data on a mobile webpage**. It follows a simple step-by-step process:

Step 1 — System Initialization

During setup(), the ESP8266 starts serial communication using Serial.begin(9600) to receive data from the Arduino. It then connects to the WiFi network using the provided SSID and password. The system continuously checks the connection status using WiFi.status() until it becomes WL_CONNECTED. Once connected, the IP address is printed, which is used to access the webpage on a phone browser.

Step 2 — Web Server Setup

A web server is created using ESP8266WebServer server(80) on port 80. The function server.on("/") defines what happens when a user opens the IP address in a browser. It generates a dynamic HTML page containing the attendance log. Basic CSS styling is added to improve readability and appearance. The webpage is set to auto-refresh every 5 seconds to show the latest updates.

Step 3 — Attendance Display

When the webpage is opened:
If no data is received yet, it shows *"No entries yet"*
Otherwise, it reads the stored attendanceLog string
Each entry is separated using
 and displayed one by one
Based on the message content:
"Entered" → shown in green (valid entry)
"Wrong" → shown in red (invalid card)
"High" → shown in orange (high temperature warning)
This makes the interface easy to understand.

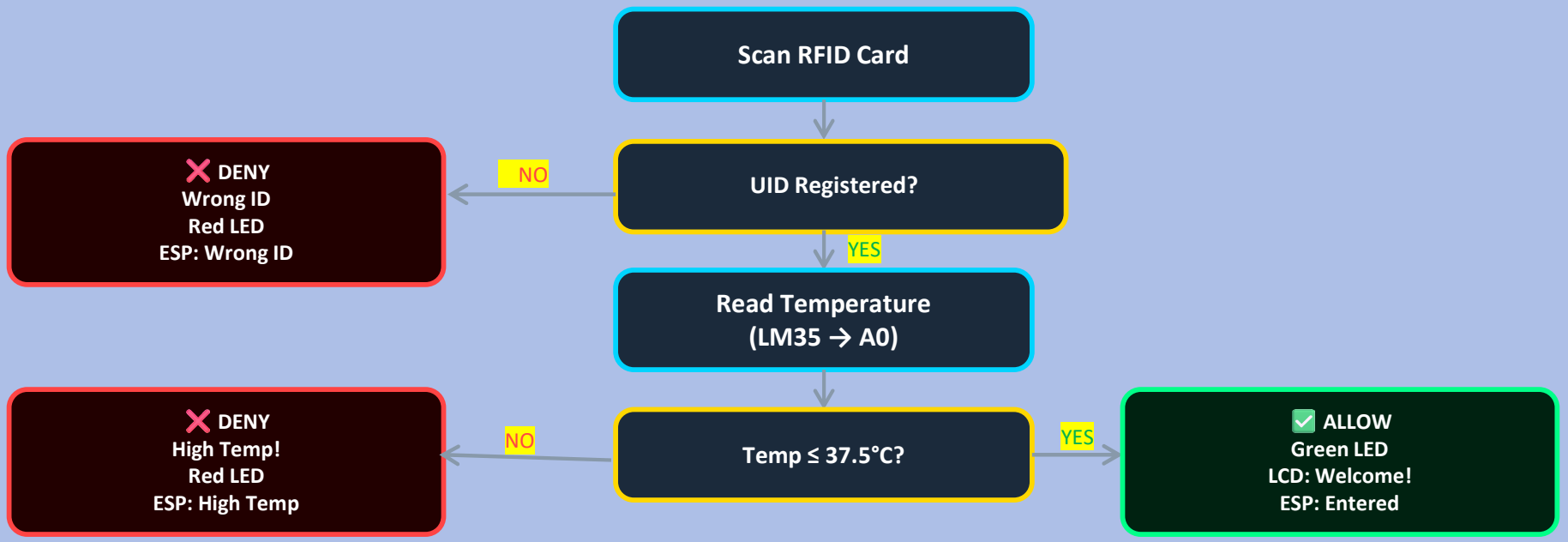
Step 4 — Data Reception from Arduino

In the loop() function, the ESP8266 continuously checks for incoming data using Serial.available(). When Arduino sends a message (like *"Person1 - 36.8C (Entered)"*), it is read using readStringUntil('\n').
The received data is cleaned using trim() and added to the attendanceLog. Each new entry is placed at the top, so the latest record appears first on the webpage.

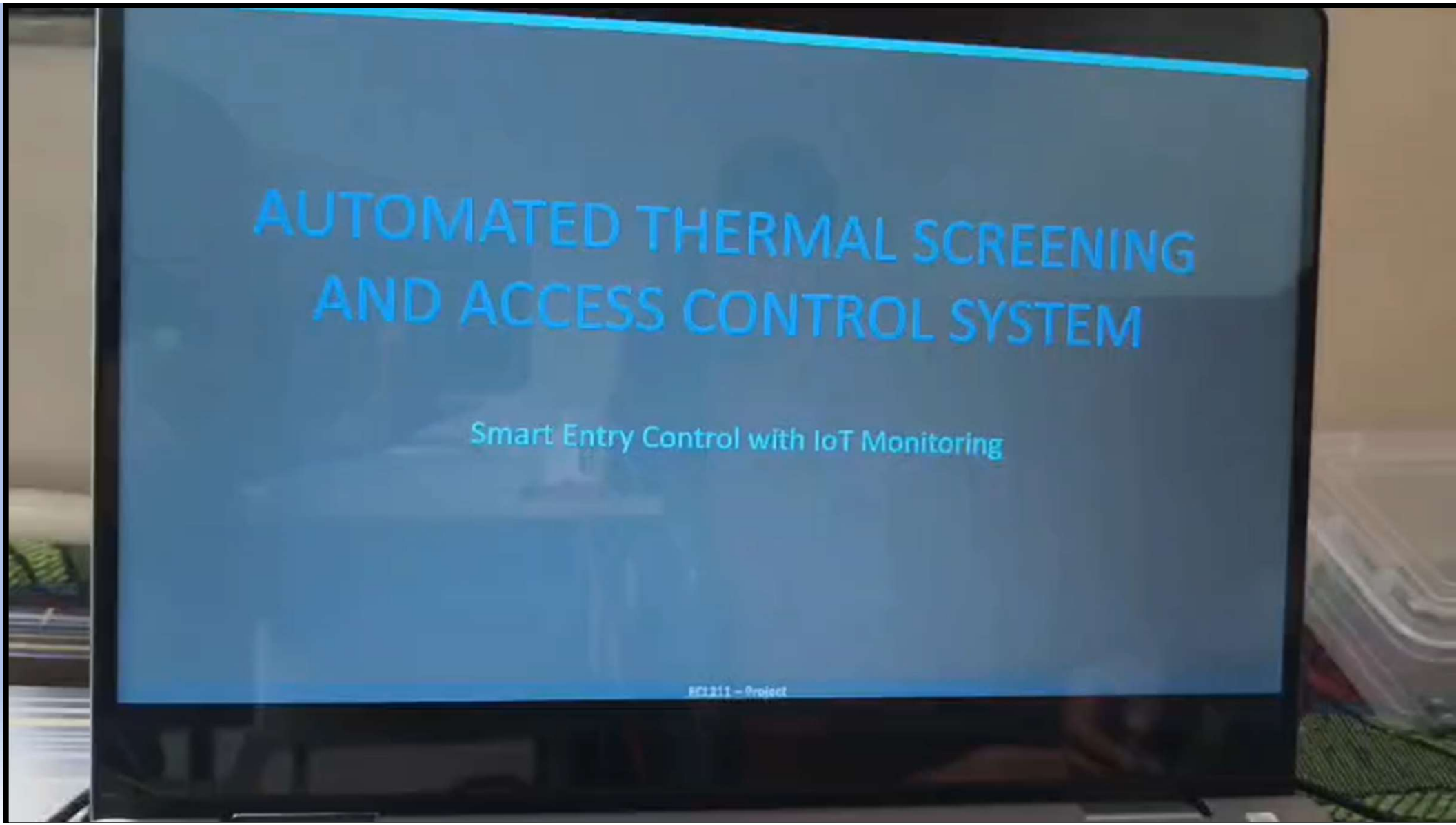
Step 5 — Continuous Web Handling

The function server.handleClient() runs continuously in the loop. It listens for incoming requests from a browser and sends the updated webpage whenever required. Due to auto-refresh, the user always sees the latest attendance records without manually refreshing.

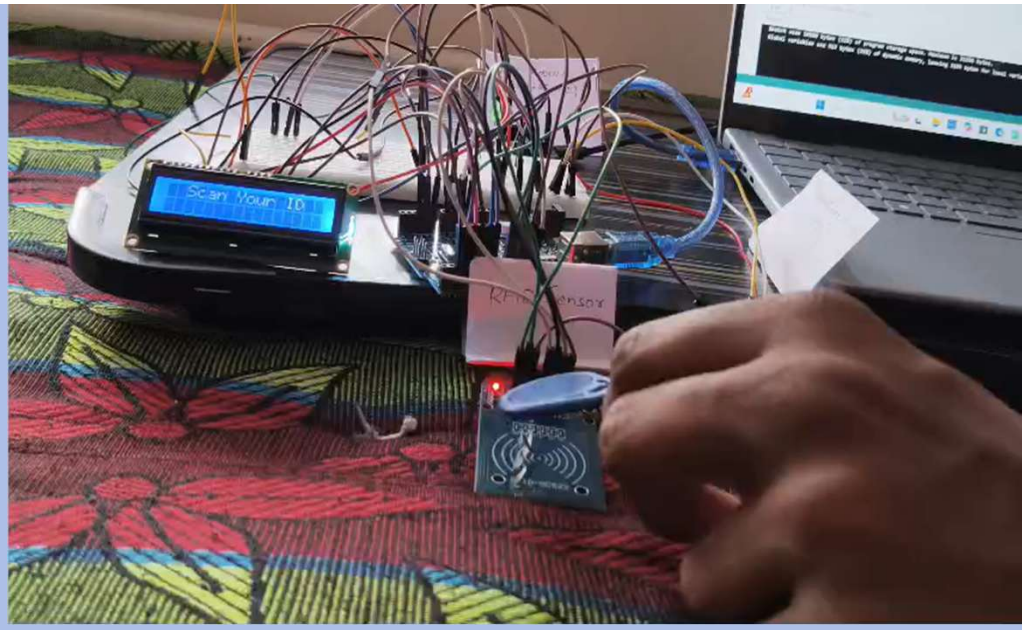
How It Works — Logic Flow

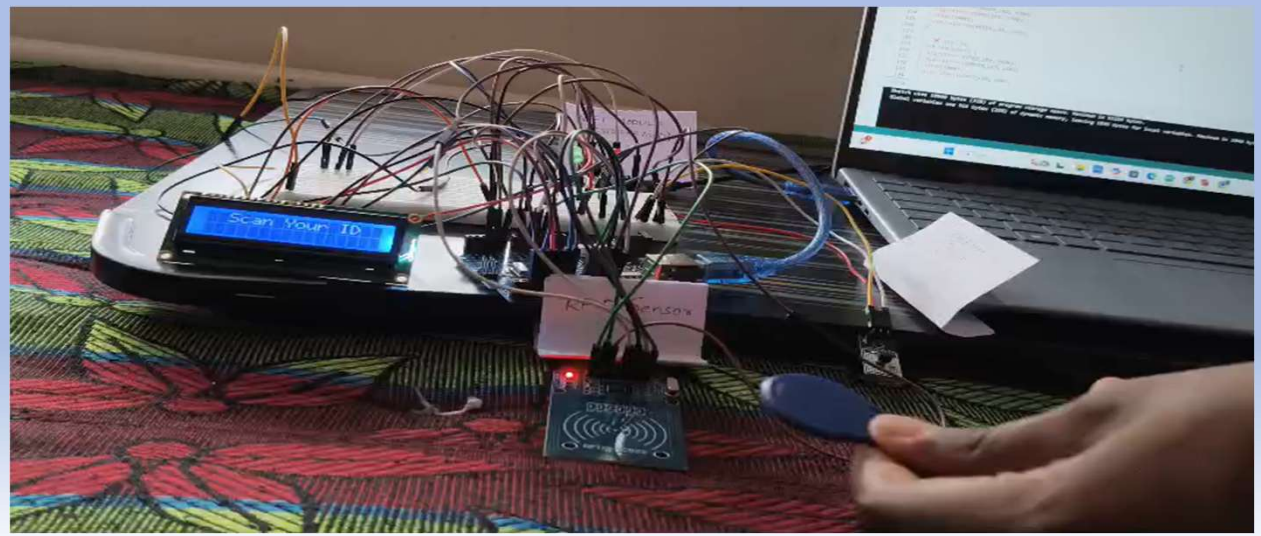


After each scan: LCD resets to 'Scan Your ID' · ESP8266 page auto-refreshes every 5 seconds

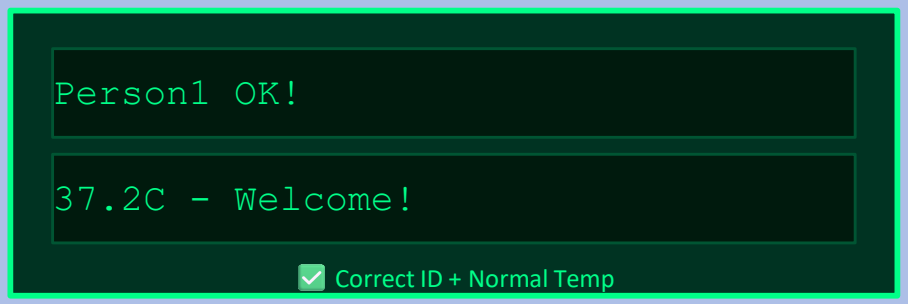

AUTOMATED THERMAL SCREENING
AND ACCESS CONTROL SYSTEM
Smart Entry Control with IoT Monitoring

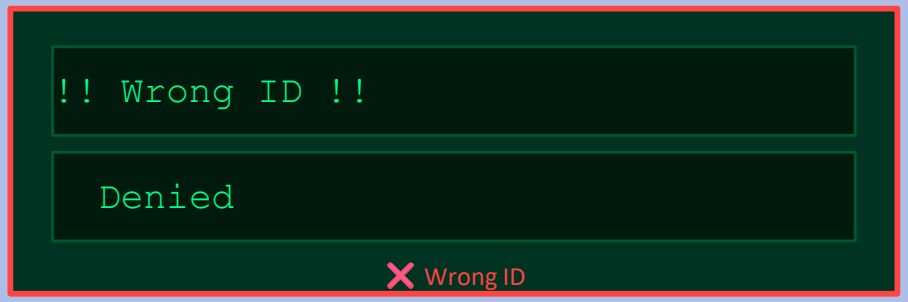

AUTOMATED THERMAL SCREENING
AND ACCESS CONTROL SYSTEM
Smart Entry Control with IoT Monitoring
EC1211 – Project

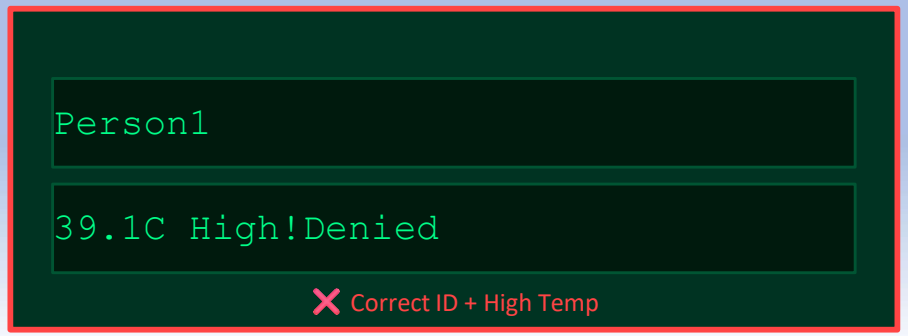

Scan Your ID

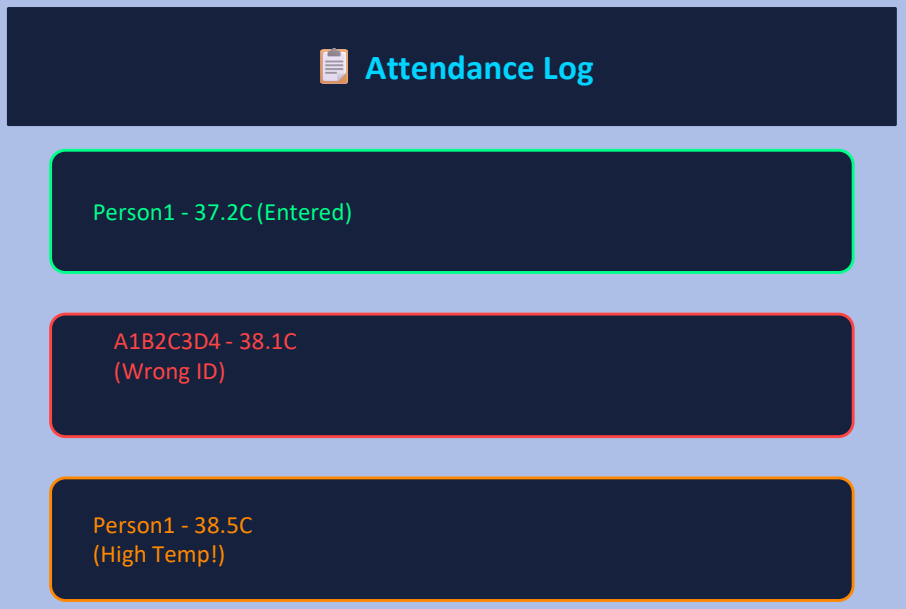

Scan Your ID

Output Display — LCD & Mobile









Project Summary

RFID Authentication

RFID-RC522 sensor scans cards. Only registered UIDs are allowed entry.

Temperature Check

LM35 reads temp via A0. Entry blocked if temp > 37.5°C.

LED Indicators

Green LED = allowed. Red LED = denied. Instant visual feedback.

LCD Display

16x2 LCD shows name, temperature and status in 4-bit mode.

IoT Monitoring

ESP8266 hosts a web page — view attendance log on our phone!

Built with Arduino Uno • RFID-RC522 • LM35 • ESP8266 • LCD 16x2

Thank You!

